

Typesetting captions with the caption package*

Axel Sommerfeldt
caption@sommerfeldt.net

2005/06/13

Abstract

The caption package provides many ways to customise the captions in floating environments such `figure` and `table` and cooperates with many other packages.¹

1 Introduction

Within the standard $\text{\LaTeX}$ classes captions haven't received the attention they deserve. Simply typeset as an ordinary paragraph there is no remarkable visual difference from the rest of the text, like here:

Figure 1: White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

There should be possibilities to change this; e.g., it would be nice if you can make the text of the caption a little bit smaller as the normal text, add an extra margin, typeset the caption label with the same font family and shape as your headings etc. Just like this one:

Figure 2: White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

With this package you can do this easily as there are many ready-to-use caption formatting options, but you are free to define your very own stuff, too.

*This package has version number v3.0f, last revised 2005/06/26.

¹A complete re-work of the user interface done together with Steven D. Cochran and Frank Mittelbach has lead to this new enhanced version 3.0.

2 Using the package

`\usepackage` Insert

```
\usepackage[<options>]{caption}[2005/06/26]
```

into the preamble of your document, i.e. the part of your document between `\documentclass` and `\begin{document}`. The options control how your captions will look like; e.g.,

```
\usepackage[margin=10pt,font=small,labelfont=bf]{caption}
```

would result in captions looking like the second one in the introduction.

`\captionsetup` For a later change of options the caption package provides the command

```
\captionsetup[<float type>]{<options>}
```

So

```
\usepackage[margin=10pt,font=small,labelfont=bf]{caption}
```

and

```
\usepackage{caption}
\captionsetup{margin=10pt,font=small,labelfont=bf}
```

are equal in their results.

It's good to know that `\captionsetup` has an effect on the current environment only. So if you want to change some settings for the current figure or table only, just place the `\captionsetup` command inside the figure or table right before the `\caption` command. For example

```
\begin{figure}
...
\captionsetup{singlelinecheck=off}
\caption{...}
\end{figure}
```

switches the single-line-check off, but only for this figure so all the other captions remain untouched.

(For a description of the optional parameter *<float type>* see section 4: “Useful stuff”.)

3 Options

3.1 Formatting

`format=` A figure or table caption mainly consists of three parts: the caption label, which says if this object is a ‘Figure’ or ‘Table’ and what number is associated with it, the caption text itself, which is normally a short description of contents, and the caption separator which separates the text from the label.

The *caption format* determines how this information will be presented; it is specified with the option

`format=<format name>` ,

having the name of the caption format as its argument.

There are two standard caption formats:²

`default` Typesets the captions as a normal paragraph. (This is the default behaviour, it is adapted from the standard L^AT_EX document classes.)

`hang` Indents the caption text, so it will ‘hang’ under the first line of the text.

An example: Specifying the option

`format=hang`

yields captions like this:

Figure 3: White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe’s finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

`indentation=` For both formats (`default` and `hang`) you can setup an extra indentation starting at the second line of the caption. You do this with the option

`indentation=<amount>`.

Three examples:

`format=default,indentation=.5cm`

Figure 4: White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe’s finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

`format=hang,indentation=-0.5cm`

Figure 5: White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe’s finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

`labelformat=` With the option

`labelformat=<label format name>`

New description
v3.0e you specify how the caption label will be typeset. There are three standard caption label formats:

`default` The caption label will be typeset as specified by the document class, usually this means the name and the number (like `simple`). (This is the default behaviour.)

²You have the option to define your own ones, too. See section 5: “*Do it yourself!*” for details.

<code>empty</code>	The caption label will be empty. This option only makes sense when used together with other options like <code>labelsep=none</code> .
<code>simple</code>	The caption label will be typeset as a name and a number.
<code>parens</code>	The number of the caption label will be typeset in parentheses.

An example: Using the options

```
labelformat=parens, labelsep=quad
```

yields captions like this one:

Figure (6) White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

`labelsep=` With the options

```
labelsep=<label separator name>
```

you specify what caption separator will be used. You can choose one of the following:

<code>none</code>	There is no caption separator. This option only makes sense when used together with other options like <code>labelformat=empty</code> .
<code>colon</code>	The caption label and text will be separated by a colon and a space. (This is the default one.)
<code>period</code>	The caption label and text will be separated by a period and a space.
<code>space</code>	The caption label and text will be separated by a single space.
<code>quad</code>	The caption label and text will be separated by a <code>\quad</code> .
<code>newline</code>	The caption label and text will be separated by a line break (<code>\</code>).

Two examples:

```
labelsep=period
```

Figure 7. White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

```
labelsep=newline, singlelinecheck=false
```

Figure 8

White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

3.2 Justification

`justification=` As addition to the caption format you could also specify a *caption justification*; it is specified with the option

```
justification=\justification name .
```

You can choose one of the following:

<code>justified</code>	Typesets the caption as a normal paragraph. (This is the default.)
<code>centering</code>	Each line of the caption will be centered.
<code>centerlast</code>	The last line of each paragraph of the caption text will be centered.
<code>centerfirst</code>	Only the first line of the caption will be centered.
<code>raggedright</code>	Each line of the caption will be moved to the left margin.
<code>RaggedRight</code>	Each line of the caption will be moved to the left margin, too. But this time the command <code>\RaggedRight</code> of the <code>ragged2e</code> package will be used to achieve this. This difference is that this time the word breaking algorithm of $\text{\TeX}$ will work inside the caption.
<code>raggedleft</code>	Each line of the caption will be moved to the right margin.

Two examples:

```
justification=centerlast
```

Figure 9: White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

```
format=hang, justification=raggedright
```

Figure 10: White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

```
labelsep=newline, justification=centering
```

Figure 11

White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

`singlelinecheck=` If the caption fit in a single line it will always be centered, ignoring the justification you set:

Figure 12: A short caption.

This behaviour is adapted from the standard L^AT_EX document classes `article`, `report`, and `book`), but using the `caption` package you can switch this special treatment of such short captions off with the option

```
singlelinecheck=<bool> .
```

Using `false`, `no`, `off` or `0` for *<bool>* you switch off the extra centering:

```
singlelinecheck=false
```

Doing so the above short caption would look like

Figure 12: A short caption.

Using `true`, `yes`, `on` or `1` for *<bool>* you switch on the extra centering again. (The default is on.)

3.3 Fonts

There are three font options which affects different parts of the caption: One affecting the whole caption (`font`), one which only affects the caption label and separator (`labelfont`) and at last one which only affects the caption text (`textfont`). You set them up using the options

```
font={ <font options> } ,
labelfont={ <font options> } and
textfont={ <font options> } .
```

And these are the available font options:

<code>scriptsize</code>	Very small size
<code>footnotesize</code>	The size usually used for footnotes
<code>small</code>	Small size
<code>normalsize</code>	Normal size
<code>large</code>	Large size
<code>Large</code>	Even larger size
<code>up</code>	Upright shape
<code>it</code>	<i>Italic shape</i>
<code>sl</code>	<i>Slanted shape</i>
<code>sc</code>	SMALL CAPS SHAPE
<code>md</code>	Medium series
<code>bf</code>	Bold series

rm	Roman family
sf	Sans Serif family
tt	Typewriter family

If you use only one of these options you can omit the braces; e.g., the options `font={small}` and `font=small` yield the same result.

Two examples:

```
font={small,it},labelfont=bf
```

Figure 13: *White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.*

```
font=small,labelfont=bf,textfont=it
```

Figure 14: *White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.*

3.4 Margins and further paragraph options

`margin=` For all captions you can specify *either* an extra margin *or* a fixed width. You do this using
`width=` the options

```
margin=<amount> or
width=<amount>
```

Nevertheless what option you use, the left and right margin will be the same.

Two examples illustrating this:

```
margin=10pt
```

Figure 15: White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

```
width=.75\textwidth
```

Figure 16: White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

`parskip=` This option is useful for captions containing more than one paragraph. It specifies the extra vertical space inserted between them:

```
parskip=<amount>
```

One example:

```
margin=10pt,parskip=5pt
```

Figure 17: First paragraph of the caption. This one contains some test, just to show how these options affect the layout of the caption.

Second paragraph of the caption. This one contains some text, too, to show how these options affect the layout of the caption.

hangindent= The option

```
hangindent=<amount>
```

is for setting up a hanging indentation starting from the second line of each paragraph. If the caption contains just a single paragraph, using this option leads to the same result as the option `indentation=` you already know about. But if the caption contains multiple paragraphs you will notice the difference:

```
format=hang,indentation=-.5cm
```

Figure 18: First paragraph of the caption. This one contains some test, just to show how these options affect the layout of the caption.

Second paragraph of the caption. This one contains some text, too, to show how these options affect the layout of the caption.

```
format=hang,hangindent=-.5cm
```

Figure 19: First paragraph of the caption. This one contains some test, just to show how these options affect the layout of the caption.

Second paragraph of the caption. This one contains some text, too, to show how these options affect the layout of the caption.

3.5 Styles

`style=` A suitable combination of caption options is called *caption style*. You can compare them more or less to page styles which you set up with `\pagestyle`: The caption style provides all settings for a whole caption layout.

You switch to an already defined caption style with the option

```
style=<style name> .
```

The caption package usually defines only the style `default` which puts all options you already know about to the default ones. This means that specifying the option

```
style=default
```

has the same effect as specifying all these options:

```
format=default, labelformat=simple, labelsep=colon,  
justification=justified, font=default, labelfont=default,  
textfont=default, margin=0pt, indentation=0pt, parindent=0pt  
hangindent=0pt, singlelinecheck=true
```

3.6 Skips

`aboveskip=` The spaces above and below the caption are controlled by the skips `\abovecaptionskip` and `\belowcaptionskip`. The standard L^AT_EX document classes `article`, `report` and `book` set `\abovecaptionskip` to 10pt and `\belowcaptionskip` to 0pt. Both skips can be changed with the command `\setlength`, but you can use these options, too:

```
aboveskip=<amount> and
belowskip=<amount> .
```

`position=` Using `\abovecaptionskip` and `\belowcaptionskip` has a major design flaw: If the caption is typeset *above* (and not *below*) the figure or table they are not set up very useful at default, because there will be some extra space above the caption but no space between the caption and the figure or table itself. (Remember: `\belowcaptionskip` is usually set to 0pt.)

Please compare the spacing in these small tables:

Table 1: A table	A B
A B	C D
C D	Table 2: A table

But you can fix this by using the option `position=`. It specifies how the spacing above and below the caption will be used:

```
position=top (or position=above)
```

tells the caption package to use the spacing useful for caption *above* the figure or table and

```
position=bottom (or position=below)
```

tells the caption package to use the spacing useful for captions *below* the figure or table. (The last one is the default setting except for `longtables`.)

So adding an extra `\captionsetup{position=top}` to the left example table gives you proper spacing around both captions:

Table 3: A table	A B
A B	C D
C D	Table 4: A table

(Technically speaking `\abovecaptionskip` and `\belowcaptionskip` will be swapped if you specify the option `position=top`, so in both cases `\abovecaptionskip` will be used between the caption and the figure or table itself.)

`tableposition=` This option is especially useful when used together with the optional argument of the `\captionsetup` command. (See section 4: “*Useful stuff*” for details.) E.g.,

```
\captionsetup[table]{position=top}
```

causes all captions within tables to be treated as captions *above* the table (regarding spacing around it). Because this is a very common setting the caption package offers an abbreviating option for the use with `\usepackage`:

New feature
v3.0a

```
\usepackage[...]{caption}
```

is equivalent to

```
\usepackage[...]{caption}  
\captionsetup[table]{position=top}
```

4 Useful stuff

`\caption` The command

```
\caption[\lst_entry]{\heading}
```

typesets the caption inside a floating environment like `figure` or `table`. Well, you already know this, but what is new is the fact then when you leave the argument *\lst_entry* empty, no entry in the list of figures or tables will be made; e.g.,

```
\caption[] {A figure without entry in the list of figures.}
```

`\caption*` The `longtable` package defines the command `\caption*` which typesets the caption without label and without entry in the list of tables. An example:

```
\begin{longtable}{cc}
  \caption*{A table}\\
  A & B \\
  C & D \\
\end{longtable}
```

looks like

A table

A	B
C	D

This package does it, too, so you can use this command now within every floating environment like `figure` or `table`, like here:

```
\begin{table}
  \caption*{A table}
  \begin{tabular}{cc}
    A & B \\
    C & D \\
  \end{table}
\end{table}
```

`\captionof` Sometimes you want to typeset a caption *outside* a floating environment, putting a figure within a `minipage` for instance. For this purpose the `caption` package offers the command

```
\captionof{\float type}[\lst_entry]{\heading}
```

Note that the first argument, the *\float type*, is mandatory here, because the `\captionof` command needs to know which name to put into the caption label (e.g. “Figure” or “Table”) and in which list to put the contents entry. An example:

```
\captionof{figure}{A figure}
\captionof{table}{A table}
```

typesets captions like this:

Figure 20: A figure

Table 6: A table

The star variant `\captionof*` has the same behaviour as the `\caption*` command: it typesets the caption without label and without entry to the list of figures or tables.

Please use both `\captionof` and `\captionof*` only *inside* environments (like `minipage` or `\parbox`), otherwise a page break can appear between content and caption. Furthermore some strange effects could occur (e.g., wrong spacing around captions).

`\ContinuedFloat` Sometimes you want to split figures or tables without giving them their own reference number. This is what the command

`\ContinuedFloat`

is for; it should be used as first command inside the floating environment. It prevents the increment of the relevant counter so a figure or table with a `\ContinuedFloat` in it gets the same reference number as the figure or table before.

An example:

```
\begin{table}
\caption{A table}
...
\end{table}
...
\begin{table}\ContinuedFloat
\caption{A table (cont.)}
...
\end{table}
```

gives the following result:

Table 7: A table

...

Table 7: A table (cont.)

`\captionsetup` We already know the `\captionsetup` command (see section 2: “Using the package”), but this time we get enlighten about the optional argument *float type*.

Remember, the syntax of this command is

```
\captionsetup[float type]{options}
```

If a *float type* gets specified, all the *options* don’t change anything at this time. Instead they only get marked for a later use, when a caption inside of a floating environment of the particular type *float type* gets typeset. For example

```
\captionsetup[figure]{options}
```

forces captions within a `figure` environment to use the given *options*.

Here comes an example to illustrate this:

```
\captionsetup{font=small}
\captionsetup[figure]{labelfont=bf}
```

gives captions like this:

Figure 21: A figure

Table 8: A table

As you see the command `\captionsetup[figure]{labelfont=bf}` only changed the font of the figure caption labels, not touching all other ones.

`\clearcaptionsetup` If you want to get rid of these parameters marked for an automatic use within a particular environment you can use the command

```
\clearcaptionsetup{<Typ>} .
```

For example `\clearcaptionsetup{figure}` would clear the extra handling in the example above:

Figure 22: A figure

Table 9: A table

As *<float type>* you can usually give one of these only two: `figure` and `table`. But as we will see later some $\text{\LaTeX}$ packages exist (like the `float` package for example) who can define additional floating environments and these two commands also works with them.

5 Do it yourself!

A family of commands is provided to allow users to define their own formats. This enables information on separators, justification, fonts, and styles to be associated with a name and kept in one place (these commands need to appear in the document preamble, this is the part between `\documentclass` and `\begin{document}`).

`\DeclareCaptionFormat` You can define your own caption formats using the command

```
\DeclareCaptionFormat{<name>}{<code using #1, #2 and #3>} .
```

At usage the system replaces `#1` with the caption label, `#2` with the separator and `#3` with the text. So the standard format default is defined inside `caption.sty` as

```
\DeclareCaptionFormat{default}{#1#2#3\par}
```

`\DeclareCaptionLabelFormat` Likewise you can define your own caption label formats:

```
\DeclareCaptionLabelFormat{<name>}{<code using #1 and #2>}
```

At usage `#1` gets replaced with the name (e.g. “figure”) and `#2` gets replaced with the reference number (e.g. “12”).

`\bothIfFirst` When you define your own caption label formats and use the `subfig` package[10], too, you
`\bothIfSecond` must take care of empty caption label names. For this purpose the commands

```
\bothIfFirst{<first arg>}{<second arg>} and
\bothIfSecond{<first arg>}{<second arg>}
```

are offered. `\bothIfFirst` tests if the first argument exists (means: is not empty), `\bothIfSecond` tests if the second argument exists. If it is so both arguments get typeset, otherwise none of them.

For example the standard label format `simple` isn't defined as

```
\DeclareCaptionLabelFormat{simple}{#1 #2} ,
```

because this could cause an extra space if #1 is empty. Instead `simple` is defined as

```
\DeclareCaptionLabelFormat{simple}{\bothIfFirst{#1}{ }#2}
,
```

causing the space to appear only if the label name is present.

`\DeclareCaptionLabelSeparator` You can define your own caption label separators with

```
\DeclareCaptionLabelSeparator{<name>}{<code>} .
```

Again an easy example taken from `caption.sty` itself:

```
\DeclareCaptionLabelSeparator{colon}{: }
```

`\DeclareCaptionJustification` You can define your own caption justifications with

```
\DeclareCaptionJustification{<name>}{<code>} .
```

The `<code>` simply gets typeset just before the caption. E.g. using the justification `raggedright`, which is defined as

```
\DeclareCaptionJustification{raggedright}{\raggedright}
,
```

yields captions with all lines moved to the left margin.

`\DeclareCaptionFont` You can define your own caption fonts with

```
\DeclareCaptionFont{<name>}{<code>} .
```

For example this package defines the options `small` and `bf` as

```
\DeclareCaptionFont{small}{\small} and
\DeclareCaptionFont{bf}{\bfseries} .
```

`\DeclareCaptionStyle` The best one comes at last: You can define your own caption styles with

```
\DeclareCaptionStyle{<name>}[<additional options>]{<options>}
```

Remember, caption styles are just a collection of suitable options, saved under a given name. You can wake up these options at any time with the option `style=<style name>`.

All caption styles are based on the default set of options. (See section 3.5: “*Styles*” for a complete list.) So you only need to specify options which are different to them.

If you specify `<additional options>` they get used in addition when the caption fits into a single line and this check was not disabled with the option `singlelinecheck=off`.

Again a very easy example taken from `caption.sty`:

```
\DeclareCaptionStyle{default}[justification=centering]{}

```

5.1 Examples

If you would like to have a colon *and* a line break as caption separator you could define it this way:

```
\DeclareCaptionLabelSeparator{period-newline}{. \\\}
```

Selecting this separator with `\captionsetup{labelsep=period-newline}` you get captions like this:

Figure 23.

White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

For short captions—which fit into one single line—this separator may not be satisfying, even when the automatically centering process is switched off (with `singlelinecheck=off`):

Figure 24.

A figure.

An own caption style which selects another caption separator automatically puts this right:

```
\DeclareCaptionStyle{period-newline}%  
[labelsep=period]{labelsep=period-newline}
```

Figure 24. A figure.

If you would like to keep the centering of these captions an appropriate definition is

```
\DeclareCaptionStyle{period-newline}%  
[labelsep=period, justification=centering]%  
{labelsep=period-newline}
```

Using this definition short captions look like

Figure 24. A figure.

while long ones still have a line break after the caption label.

Slightly changed, you also get centered captions if they are longer than one line:

```
\DeclareCaptionStyle{period-newline}%  
[labelsep=period]%  
{labelsep=period-newline, justification=centering}
```

Figure 25.

White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

Another example: You want captions to look like this:

White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

(Figure 26)

You could do it this way:

```
\DeclareCaptionFormat{reverse}{#3#2#1}
\DeclareCaptionLabelFormat{fullparens}{(\bothIfFirst{#1}{ }#2)}
\DeclareCaptionLabelSeparator{fill}{\hfill}
\captionsetup{format=reverse,labelformat=fullparens,
              labelsep=fill,font=small,labelfont=it}
```

Another example: The caption text should go into the left margin; a possible solution would be:

```
\DeclareCaptionFormat{llap}{\llap{#1#2}#3\par}
\captionsetup{format=llap,labelsep=quad,singlelinecheck=no}
```

As a result you would get captions like this:

Figure 27 White sand beaches. The pink smoothness of the conch shell. A sea abundant with possibilities. Duty-free shops filled with Europe's finest gifts and perfumes. Play your favorite game of golf amidst the tropical greens on one of the many championship courses.

6 Using non-standard document classes

New description
v3.0d

The caption package was developed using the standard document classes `article`, `report` and `book`.

If you would like to use the caption package with the KOMA-Script classes or with the memoir class, you have to take into consideration that all the possibilities for customization of the captions the KOMA-Script classes or memoir class have to offer will get lost. (And they have a lot of possibilities to offer!) So class options like `tablecaptionabove` and commands like `\captionabove`, `\captionbelow`, `\captionformat`, `\figureformat`, `\tableformat`, `\setcapindent`, `\setcaphanging`, `\captionstyle` etc. will not work anymore. So make a wise decision!

Using the caption package together with document classes not mentioned so far is not recommended at the moment – unwanted layout changes, side effects or failures could occur. (But future versions of the caption package will contain adaptations for more document classes!)

7 Using other packages

The caption package contains special adaptations to other packages who handle with captions, too, so the captions always should look like you have specified them to look like.

These are the packages the caption package is adapted to:

	<code>float</code>	Gives you the possibility to define new floating environments
	<code>hypcap</code>	Adjusting <code>hyperref</code> anchors of captions
	<code>listings</code>	Typesets source code listings
	<code>longtable</code>	Typesets tables spanned over multiple pages
	<code>rotating</code>	Supports rotated figures and tables
	<code>sidecap</code>	Offers captions <i>beside</i> figures or tables
	<code>supertabular</code>	Typesets tables spanned over multiple pages
New feature v3.0b	If you use one of the above packages together with the caption package you get the additional possibility to set up captions with	

```
\captionsetup[<environment>]{<options>}
```

These options will apply for captions inside these environments automatically. For example

```
\captionsetup[lstlisting]{labelfont=bf}
```

forces captions inside the `lstlisting` environment to have bold labels. (Please note that this do not work with the `sideways` environments offered by the `rotating` package.)

If a certain support is not desired you can switch it off using the caption package option

```
\usepackage[... ,<package>=no]{caption}
```

For example specifying the option `float=no` means you don't like the caption package to support the float package. (Note: You can specify these options only within the `\usepackage` command, especially *not* at a later time with `\captionsetup`.)

For further information about the supported packages please take a look at the documentation belonging to it or buy yourself The $\text{\LaTeX}$ Companion[1].

7.1 The float package

A very useful feature is provided by the float package[2]: It offers the float placement specifier `H` which is much more restrictive than the specifier `h` offered by $\text{\LaTeX}$. While the latter one is only a recommendation to $\text{\LaTeX}$ to set the float “here”, the `H` forces the float to appear exactly at the spot where it occurs in your input file and nowhere else.

Furthermore it offers different styles for floating environments, these styles are `plain`, `plaintop`, `ruled`, and `boxed`. You can link one of these styles to either new floating environments or to one of the existing environments `figure` and `table`.

If you are using the caption package together with the float package this caption style called `ruled` gets defined automatically:

```
\DeclareCaptionStyle{ruled}{labelfont=bf,labelsep=space}
```

This style represents the caption layout in `ruled` styled floats. For you as an end user this means that captions within `ruled` floats will always look like this, nevertheless what generic caption options do you specify:

Program 7.1 The first program. This hasn't got anything to do with the package but is included as an example. Note the `ruled` float style.

```
#include <stdio.h>

int main(int argc, char **argv)
{
    for (int i = 0; i < argc; ++i)
        printf("argv[%d] = %s\n", i, argv[i]);
    return 0;
}
```

If you want a different layout for `ruled` captions you have to define your own one using the command

```
\DeclareCaptionStyle{ruled}{\options} .
```

This mechanism also works with all other float styles. If you want a special caption layout for `plain` or `boxed` floats for example you can simply define a suitable caption style with the same name as the float style.

Note: For successful cooperation you need the float package version 1.3 or newer.

7.2 The listings package

New description
v3.0b

The `listings` package[6] is a source code printer for $\text{\LaTeX}$. You can typeset stand alone files as well as listings with an environment similar to `verbatim` as well as you can print code snippets using a command similar to `\verb`. Many parameters control the output and if your preferred programming language isn't already supported, you can make your own definition.

Note: For successful cooperation you need the `listings` package version 1.2 or higher. You'll get an error message when using an older version!

7.3 The longtable package

The `longtable` package[7] offers the environment `longtable` which behaves similar to the `tabular` environment, but the table itself can span multiple pages.

Note: For successful cooperation you need the `longtable` package version 3.15 or newer.

7.4 The rotating package

The `rotating` package[8] offers the floating environments `sidewaysfigure` and `sideways-table` which are just like normal figures and tables but rotated by 90 degree. Furthermore they always use a full page on their own.

7.5 The sidecap package

New description
v3.0b

The `sidecap` package[9] offers the floating environments `SCfigure` and `SCtable`

which are like normal figures and tables but the caption will be put *beside* the contents. The `sidecap` package offers it's own options for justification. If set, they will override the one specified with the caption option `justification=` for captions beside their contents.

`listof=` Using the `sidecap` package you will probably notice that suppressing the entry in the list of figures or tables with `\caption[]{\dots}` won't work inside these environments. This is caused by the implementation design of the `sidecap` package, but you can use `\captionsetup{listof=false}` inside the figure or table as an alternative here.

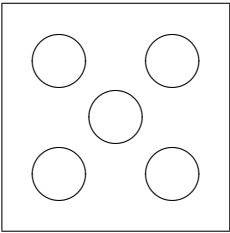


Figure 28: A small example with the caption beside the figure.

7.6 The supertabular package

The `supertabular` package[11] offers the environment `supertabular` which is quite similar to the `longtable` environment provided by the `longtable` package. Both offers the typesetting of tables which can span multiple pages. For a detailed discussion about the differences between these powerful packages please take a look at The L^AT_EX Companion[1].

7.7 Known incompatibilities

New description v3.0b Using the caption package together with one of the following packages is not recommended; usually this would cause unwanted side effects or even errors:

`ccaption`, `hvfloat`, `nonfloat`

8 Compatibility to older versions

8.1 caption version 1.x

This version of the `caption` package still supports the old options and commands provided by the version 1.x of this package. So there shouldn't occur any problems compiling old documents, but please don't mix old options and commands with the new ones. This isn't supported and can yield to ugly side effects.

Here comes a short oversight of the old options and commands and how they are replaced within this version of the `caption` package:

caption 1.x	caption 3.x
<code>normal</code>	<code>format=default</code>
<code>hang</code>	<code>format=hang</code>
<code>isu</code>	<code>format=hang</code>

caption 1.x	caption 3.x
center	justification=centering
centerlast	justification=centerlast
anne	justification=centerlast
nooneline	singlelinecheck=off
scriptsize	font=scriptsize
footnotesize	font=footnotesize
small	font=small
normalsize	font=normalsize
large	font=large
Large	font=Large
up	labelfont=up
it	labelfont=it
sl	labelfont=sl
sc	labelfont=sc
md	labelfont=md
bf	labelfont=bf
rm	labelfont=rm
sf	labelfont=sf
tt	labelfont=tt
\setlength{\captionmargin}	margin= <i>(amount)</i>
\renewcommand{\captionfont}	\DeclareCaptionFont + \captionsetup{font= <i>(name)</i> }
\renewcommand{\captionsize}	\DeclareCaptionFont + \captionsetup{font= <i>(name)</i> }
\renewcommand{\captionlabelfont}	\DeclareCaptionLabelFont + \captionsetup{labelfont= <i>(name)</i> }

8.2 caption2 version 2.x

Although they do very similar stuff the packages `caption` and `caption2` have a very different implementation design. So this version of the `caption` package isn't compatible to the `caption2` package at all. Of course for compiling old documents you can still use the `caption2` package, the latest version is provided with this package. But newly created documents shouldn't use the `caption2` package, please use the `caption` package instead as described in this manual.

9 Further reading

I recommend the following documents for further reading:

- The `TEX` FAQ - Frequently asked questions about `TEX` and `LATEX`:

<http://faq.tug.org/>

- A French FAQ can be found at

<http://www.grappa.univ-lille3.fr/FAQ-LaTeX/>

- `epslatex` from Keith Reckdahl contains many tips around graphics in `LATEX 2ε`. You will find this document in the directory

`ftp://ftp.ctan.org/pub/tex/info/`
as `epslatex.ps` and `epslatex.pdf`.
There is also a french translation available:
`ftp://ftp.ctan.org/pub/tex/info/fepslatex.ps`

10 Thanks

I would like to thank Katja Melzner, Steven D. Cochran, Frank Mittelbach, David Carlisle, Carsten Hinz, and Olga Lapko. Thanks a lot for all your help, ideas, patience, spirit, and support!

Also I would like to thank Harald Harders, Peter Löffler, Peng Yu, Alexander Zimmermann, Matthias Pospiech, Jürgen Wieferink, Christoph Bartoschek, and Uwe Stöhr who all helped to make this package a better one.

11 The Implementation

The caption package consists of two parts – the kernel and the main package.

The kernel provides all the user commands and internal macros which are necessary for typesetting captions and setting parameters regarding these. While the standard $\LaTeX$ document classes provides an internal command called `\@makecaption` and no options to control its behavior (except the vertical skips above and below them), we provide similar commands called `\caption@make` and `\caption@@make`, but with a lot of options. Loading the kernel part do not change the output of a $\LaTeX$ document – it just provides functionality which can be used by $\LaTeX 2\epsilon$ packages which typesets captions, like the caption package or the subfig package.

The caption package itself redefines the $\LaTeX$ commands `\caption`, `\@caption`, and `\@makecaption` and maps the latter one to `\caption@@make`, giving the user the possibility to control the captions of the floating environments `figure` and `table`. Furthermore it does similar to the caption stuff coming from other packages like the `longtable` package: Mapping the appropriate internal commands (like `\LT@makecaption`) to the ones offered by the caption kernel. So you can think of the caption package as a layer package, it simply provides adaption layers between the caption stuff coming from $\LaTeX$ itself or any $\LaTeX 2\epsilon$ package and the caption stuff offered by the caption kernel.

11.1 Kernel

Identification

```
1 \NeedsTeXFormat{LaTeX2e}[1994/12/01]
2 \ProvidesPackage{caption3}[2005/06/11 v3.0f caption3 kernel (AS)]
```

Generic helpers

`\@nameundef` This is the opposite part to `\@namedef` which is offered by the $\LaTeX$ kernel. We use it to remove the definition of some commands and keyval options after `\begin{document}` (to save $\TeX$ memory) or to remove caption options defined with `\captionsetup[⟨type⟩]`.

```
3 \providecommand*\@nameundef[1]{%
4   \expandafter\let\csname #1\endcsname\@undefined}
```

`\l@addto@macro` The $\LaTeX 2\epsilon$ kernel offers the internal helper macro `\g@addto@macro` which globally adds commands to any existing macro, like in `\AtBeginDocument`. This is the same but it works local, not global.

```
5 \providecommand\l@addto@macro[2]{%
6   \begingroup
7     \toks@\expandafter{#1#2}%
8     \edef\@tempa{\endgroup\def\noexpand#1{\the\toks@}}%
9   \@tempa}
```

`\bothIfFirst` `\bothIfSecond` `\bothIfFirst` tests if the first argument is not empty, `\bothIfSecond` tests if the second argument is not empty. If yes both arguments get typeset, otherwise none of them.

```
10 \def\bothIfFirst#1#2{%
11   \protected@edef\caption@tempa{#1}%
12   \ifx\caption@tempa\empty\else
13     #1#2%
14   \fi}
15 \def\bothIfSecond#1#2{%
```

```

16 \protected@edef\caption@tempa{#2}%
17 \ifx\caption@tempa\@empty\else
18   #1#2%
19 \fi}

```

`\caption@ifinlist` This helper macro checks if the first argument is in the list which is offered as second argument. So for example

```
\caption@ifinlist{frank}{axel, frank, steven}{yes}{no}
```

would expand to yes.

```

20 \def\caption@ifinlist#1#2{%
21   \let\next\@secondoftwo
22   \edef\caption@tempa{#1}%
23   \@for\caption@tempb:={#2}\do{%
24     \ifx\caption@tempa\caption@tempb
25       \let\next\@firstoftwo
26     \fi}%
27   \next}

```

`\caption@setbool` For setting and testing boolean options we offer these two helper macros:

```

\caption@ifbool
\caption@setbool{<name>}{<value>}
                    (with value = false/true/no/yes/off/on/0/1)
\caption@ifbool{<name>}{<if-clause>}{<else-clause>}

```

```

28 \def\caption@setbool#1#2{%
29   \caption@ifinlist{#2}{1,true,yes,on}{%
30     \expandafter\let\csname caption@if#1\endcsname\@firstoftwo
31   }{\caption@ifinlist{#2}{0,false,no,off}{%
32     \expandafter\let\csname caption@if#1\endcsname\@secondoftwo
33   }}%
34   \PackageError{caption}{Undefined boolean value `#2'}{\caption@eh}%
35   }}
36 \def\caption@ifbool#1{\@nameuse{caption@if#1}}

```

Using the keyval package

We need the keyval package for option handling, so we load it here.

```
37 \RequirePackage{keyval}[1997/11/10]
```

`\undefine@key` This helper macro is the opposite of `\define@key`, it removes a keyval definition.

```

38 \providecommand*\undefine@key[2]{%
39   \@nameundef{KV@#1@#2}\@nameundef{KV@#1@#2@default}}

```

`\DeclareCaptionOption` `\DeclareCaptionOption{<option>}{<code>}`
`\DeclareCaptionOption*{<option>}{<code>}`

The starred form makes the option only available during the lifetime of the current package, so they are only avail at `\usepackage`, but can't be used with `\captionsetup` later on.

```

40 \newcommand\DeclareCaptionOption{%
41   \@ifstar{\caption@declareoption\AtEndOfPackage}%
42   {\caption@declareoption\@gobble}}

```

```

43 \newcommand*\caption@declareoption[2]{%
44   #1{\undefine@key{caption}{#2}}\define@key{caption}{#2}}
45 \@onlypreamble\DeclareCaptionOption
46 \@onlypreamble\caption@declareoption

\captionsetup \captionsetup[<type>]{<keyval-list of options>}
If 'type' is set, we simply save or append the option list, otherwise we 'execute' it with
\setkeys.
47 \def\captionsetup{\@ifnextchar[\caption@setuptype\caption@setup}
48 \def\caption@setuptype[#1]#2{%
49   \@ifundefined{caption@typ@#1}%
50     {\@namedef{caption@typ@#1}{#2}}%
51     {\expandafter\l@addto@macro\csname caption@typ@#1\endcsname{, #2}}}
52 \def\caption@setup{\setkeys{caption}}

\caption@settype \caption@settype{<type>}
Caption options which have been saved with \captionsetup[<type>] can be executed
using this macro. (It simply executes the saved option list, if there is any.)
53 \def\caption@settype#1{%
54   \@ifundefined{caption@typ@#1}{}{%
55     \caption@esetup{\csname caption@typ@#1\endcsname}}}%

\caption@esetup To execute a keyval-list of options saved within a macro we need this special version of
\caption@setup which expands the argument first.
56 \def\caption@esetup#1{%
57   \edef\caption@tempa{\noexpand\caption@setup{#1}}%
58   \caption@tempa}

\clearcaptionsetup \clearcaptionsetup{<type>}
This removes the saved option list associated with <type>.
59 \newcommand*\clearcaptionsetup[1]{\@nameundef{caption@typ@#1}}

\showcaptionsetup \showcaptionsetup[<package>]{<type>}
This command is for debugging issues: It shows the saved option list which is associated
with <type>.
60 \newcommand*\showcaptionsetup[2][\@firstofone]{%
61   \GenericWarning{}{%
62     #1 Caption Info: KV list on `#2'\MessageBreak
63     #1 Caption Data: (%
64     \@ifundefined{caption@typ@#2}{%
65       % Empty -- print nothing.
66     }{%
67       \@nameuse{caption@typ@#2}%
68     }%
69   )}}

```

Errors

```

\caption@eh We only provide this simple error message as helper for the user.
70 \newcommand\caption@eh{%
71   If you do not understand this error, please take a closer look\MessageBreak
72   at the documentation of the 'caption' package.\MessageBreak
73   \@ehc}

```

Margin resp. width

```
\captionmargin \captionmargin and \captionwidth contains the extra margin resp. the total
\captionwidth width used for captions. Never set these values in a direct way, they are just accessible in
\ifcaption@width user documents to provide compatibility to caption.sty v1.x.

74 \newdimen\captionmargin
75 \newdimen\captionwidth
76 \newif\ifcaption@width

77 \DeclareCaptionOption{margin}{\caption@setmargin{#1}}
78 \DeclareCaptionOption{width}{\caption@setwidth{#1}}

\caption@setmargin Note that we can only set one at a time, 'margin' or 'width'. Which dimension is actually
\caption@setwidth set will be recognized by \ifcaption@width.

79 \newcommand\caption@setmargin{%
80   \caption@widthfalse
81   \setlength\captionmargin{
82 \newcommand\caption@setwidth{%
83   \caption@widthtrue
84   \setlength\captionwidth{
```

Indentions

```
\captionindent These are the indentions we support.
\captionparindent 85 \newdimen\captionindent
\captionhangindent 86 \newdimen\captionparindent
87 \newdimen\captionhangindent

88 \DeclareCaptionOption{indent}[\leftmargini]{\setlength\captionindent{#1}}% obsolete
89 \DeclareCaptionOption{indention}[\leftmargini]{\setlength\captionindent{#1}}
90 \DeclareCaptionOption{parindent}[\parindent]{\setlength\captionparindent{#1}}% changed
91 \DeclareCaptionOption{hangindent}[0pt]{\setlength\captionhangindent{#1}}% changed
```

Styles

```
\DeclareCaptionStyle \DeclareCaptionStyle{<name>}[<single-line-list-of-KV>]{<list-of-KV>}

92 \newcommand*\DeclareCaptionStyle[1]{%
93   \@ifnextchar[{\caption@declarestyle{#1}}{\caption@declarestyle{#1} []}
94 \def\caption@declarestyle#1[#2]#3{%
95   \global\@namedef{caption@sls@#1}{#2}%
96   \global\@namedef{caption@sty@#1}{#3}}
97 \@onlypreamble\DeclareCaptionStyle
98 \@onlypreamble\caption@declarestyle

99 \DeclareCaptionOption{style}{\caption@setstyle{#1}}

\caption@setstyle \caption@setstyle{<name>}
\caption@setstyle*{<name>}
```

Selecting a caption style means saving the additional *<single-line-list-of-KV>* (this will be done by `\caption@sls`), resetting the caption options to the default ones (this will be done using `\caption@setdefault`) and executing the *<list-of-KV>* options (this will be done using `\caption@setup`).

The starred version will give no error message if the given style is not defined.

```

100 \newcommand\caption@setstyle{%
101   \@ifstar{\caption@@setstyle\@gobble}{\caption@@setstyle\@firstofone}}
102 \newcommand*\caption@@setstyle[2]{%
103   \@ifundefined{caption@sty@#2}%
104     {#1{\PackageError{caption}{Undefined caption style `#2'}{\caption@eh}}}%
105 {\expandafter\let\expandafter\caption@sls\csname caption@sls@#2\endcsname
106   \caption@setdefault\caption@esetup{\csname caption@sty@#2\endcsname}}}

```

`\caption@setdefault` This resets (nearly) all caption options to the default ones. (Note that this does not touch the skips and the positioning!)

```

107 \newcommand\caption@setdefault{\captionsetup{%
108   format=default,labelformat=default,labelsep=default,justification=default,%
109   font=default,labelfont=default,textfont=default,%
110   margin=0pt,indentation=0pt,parindent=0pt,hangindent=0pt,singlelinecheck}}

```

There is only one pre-defined style, called ‘default’. It’s a perfect match to the standard L^AT_EX document classes: If the caption fits in one single line, it is typeset centered.

```

111 \DeclareCaptionStyle{default}[indent=0pt,justification=centering]{}

```

Formats

```

\DeclareCaptionFormat \DeclareCaptionFormat{<name>}{<code with #1, #2, and #3>}
\DeclareCaptionFormat*{<name>}{<code with #1, #2, and #3>}

```

The starred form causes the code being typeset in vertical (instead of horizontal) mode, but does not support the `indentation=` option.

```

112 \def\DeclareCaptionFormat{%
113   \@ifstar{\caption@declareformat\@gobble}{\caption@declareformat\@firstofone}}
114 \newcommand\caption@declareformat[3]{%
115   \global\expandafter\let\csname caption@ifh@#2\endcsname#1%
116   \global\long\expandafter\def\csname caption@fmt@#2\endcsname##1##2##3{#3}}
117 \@onlypreamble\DeclareCaptionFormat
118 \@onlypreamble\caption@declareformat

119 \DeclareCaptionOption{format}{\caption@setformat{#1}}

```

`\caption@setformat` `\caption@setformat{<name>}`

Selecting a caption format simply means saving the code (in `\caption@fmt`) and if the code should be used in horizontal or vertical mode (`\caption@ifh`).

```

120 \newcommand*\caption@setformat[1]{%
121   \@ifundefined{caption@fmt@#1}%
122     {\PackageError{caption}{Undefined caption format `#1'}{\caption@eh}}%
123     {\expandafter\let\expandafter\caption@ifh\csname caption@ifh@#1\endcsname
124     \expandafter\let\expandafter\caption@fmt\csname caption@fmt@#1\endcsname}}

```

There are two pre-defined formats, called ‘@normal’ and ‘hang’.

```

125 \DeclareCaptionFormat{@normal}{#1#2#3\par}
126 \DeclareCaptionFormat{hang}{%
127   \@hangfrom{#1#2}%
128   \advance\captionparindent\hangindent
129   \advance\captionhangindent\hangindent
130   \caption@@par
131   #3\par}

```

‘default’ usually maps to ‘@normal’.

```
132 \def\caption@fmt@default{\caption@fmt@@normal}  
133 \def\caption@ifh@default{\caption@ifh@@normal}% bugfix v3.0e (05-04-28)
```

Label formats

```
DeclareCaptionLabelFormat \DeclareCaptionLabelFormat{<name>}{<code with #1 and #2>}  
134 \newcommand*\DeclareCaptionLabelFormat[2]{%  
135   \global\expandafter\def\csname caption@lfmt@#1\endcsname##1##2{#2}}  
136 \@onlypreamble\DeclareCaptionLabelFormat  
  
137 \DeclareCaptionOption{labelformat}{\caption@setlabelformat{#1}}
```

```
\caption@setlabelformat \caption@setlabelformat{<name>}
```

Selecting a caption label format simply means saving the code (in \caption@lfmt).

```
138 \newcommand*\caption@setlabelformat[1]{%  
139   \@ifundefined{caption@lfmt@#1}%  
140     {\PackageError{caption}{Undefined caption label format `#1'}{\caption@eh}}%  
141     {\expandafter\let\expandafter\caption@lfmt\csname caption@lfmt@#1\endcsname}}
```

There are three pre-defined label formats, called ‘empty’, ‘simple’, and ‘parens’.

```
142 \DeclareCaptionLabelFormat{empty}{}  
143 \DeclareCaptionLabelFormat{simple}{\bothIfFirst{#1}{\nobreakspace}#2}  
144 \DeclareCaptionLabelFormat{parens}{\bothIfFirst{#1}{\nobreakspace}{#2}}
```

‘default’ usually maps to ‘simple’.

```
145 \def\caption@lfmt@default{\caption@lfmt@simple}
```

Label separators

```
lareCaptionLabelSeparator \DeclareCaptionLabelSeparator{<name>}{<code>}  
146 \newcommand\DeclareCaptionLabelSeparator[2]{%  
147   \global\long\@namedef{caption@lsep@#1}{#2}}  
148 \@onlypreamble\DeclareCaptionLabelSeparator  
  
149 \DeclareCaptionOption{labelsep}{\caption@setlabelseparator{#1}}  
150 \DeclareCaptionOption{labelseparator}{\caption@setlabelseparator{#1}}
```

```
caption@setlabelseparator \caption@setlabelseparator{<name>}
```

Selecting a caption label separator simply means saving the code (in \caption@lsep).

```
151 \newcommand*\caption@setlabelseparator[1]{%  
152   \@ifundefined{caption@lsep@#1}%  
153     {\PackageError{caption}{Undefined caption label separator `#1'}{\caption@eh}}%  
154     {\expandafter\let\expandafter\caption@lsep\csname caption@lsep@#1\endcsname}}
```

There are six pre-defined label separators, called ‘none’, ‘colon’, ‘period’, ‘space’, ‘quad’, and ‘newline’.

```
155 \DeclareCaptionLabelSeparator{none}{}  
156 \DeclareCaptionLabelSeparator{colon}{: }  
157 \DeclareCaptionLabelSeparator{period}{. }  
158 \DeclareCaptionLabelSeparator{space}{ }  
159 \DeclareCaptionLabelSeparator{quad}{\quad}  
160 \DeclareCaptionLabelSeparator{newline}{\\}% 05-03-23 (v3.0f)
```

‘default’ usually maps to ‘colon’.

```
161 \def\caption@lsep@default{\caption@lsep@colon}
```

Justifications

```
\DeclareCaptionJustification \DeclareCaptionJustification{<name>}{<code>}
162 \newcommand*\DeclareCaptionJustification[2]{%
163   \global\@namedef{caption@hj@#1}{#2}}
164 %\newcommand\DeclareCaptionJustification{\DeclareCaptionFont}
165 \@onlypreamble\DeclareCaptionJustification

166 \DeclareCaptionOption{justification}{\caption@setjustification{#1}}
```

```
\caption@setjustification \caption@setjustification{<name>}
```

Selecting a caption justification simply means saving the code (in \caption@hj).

```
167 \newcommand*\caption@setjustification[1]{%
168   \@ifundefined{caption@hj@#1}%
169     {\PackageError{caption}{Undefined caption justification ‘#1’}{\caption@eh}}%
170     {\expandafter\let\expandafter\caption@hj\csname caption@hj@#1\endcsname}}
171 %\newcommand\caption@setjustification{\caption@setfont{@hj}}
```

These are the pre-defined justification code snippets.

```
172 \DeclareCaptionJustification{justified}{}
173 \DeclareCaptionJustification{centering}{\centering}
174 \DeclareCaptionJustification{centerfirst}{\caption@centerfirst}
175 \DeclareCaptionJustification{centerlast}{\caption@centerlast}
176 \DeclareCaptionJustification{raggedleft}{\raggedleft}
177 \DeclareCaptionJustification{raggedright}{\raggedright}
```

‘default’ usually maps to ‘justified’.

```
178 \def\caption@hj@default{\caption@hj@justified}
```

\caption@centerfirst Please blame Frank Mittelbach for \caption@centerfirst and Anne Brüggemann-Klein for \caption@centerlast :-)

```
179 \newcommand\caption@centerfirst{%
180   \edef\caption@normaladjust{%
181     \leftskip\the\leftskip
182     \rightskip\the\rightskip
183     \parfillskip\the\parfillskip\relax}%
184   \leftskip\z@\@plus -1fil%
185   \rightskip\z@\@plus 1fil%
186   \parfillskip\z@skip
187   \noindent\hskip\z@\@plus 2fil%
188   \@setpar{\@par\@restorepar\caption@normaladjust}}
189 \newcommand\caption@centerlast{%
190   \leftskip\z@\@plus 1fil%
191   \rightskip\z@\@plus -1fil%
192   \parfillskip\z@\@plus 2fil\relax}
```

We also support the upper-case commands offered by the ragged2e package. Note that these just map to their lower-case variants if the ragged2e package is not available.

```
193 \DeclareCaptionJustification{Centering}{%
194   \caption@ragged\Centering\centering}
```

```

195 \DeclareCaptionJustification{RaggedLeft}{%
196   \caption@ragged\RaggedLeft\raggedleft}
197 \DeclareCaptionJustification{RaggedRight}{%
198   \caption@ragged\RaggedRight\raggedright}

```

`\caption@ragged` `\caption@ragged` will be basically defined as

```

\AtBeginDocument{\IfFileExists{ragged2e.sty}%
  {\RequirePackage{ragged2e}\let\caption@ragged\@firstoftwo}%
  {\let\caption@ragged\@secondoftwo}}

```

but with a warning if the `ragged2e` package is not avail. (This warning will by typeout only ones per option, that's why we need the `\caption\string#1` stuff.)

```

199 \newcommand*\caption@ragged[2]{%
200   \@ifundefined{caption\string#1}{%
201     \PackageWarning{caption}{%
202       Cannot locate the 'ragged2e' package, therefore\MessageBreak
203       substituting \string#2 for \string#1\MessageBreak}%
204     \global\@namedef{caption\string#1}}{}%
205   #2}

206 \AtBeginDocument{\IfFileExists{ragged2e.sty}{%
207   \RequirePackage{ragged2e}\let\caption@ragged\@firstoftwo}}{}

```

Fonts

```

\DeclareCaptionFont \DeclareCaptionFont{<name>}{<code>}

208 \newcommand\DeclareCaptionFont[2]{%
209   \define@key{caption@fnt}{#1}[]{\g@addto@macro\caption@tempa{#2}}
210 \@onlypreamble\DeclareCaptionFont

211 \DeclareCaptionOption{font}{\caption@setfont{font}{#1}}
212 \DeclareCaptionOption{labelfont}{\caption@setfont{labelfont}{#1}}
213 \DeclareCaptionOption{textfont}{\caption@setfont{textfont}{#1}}

```

`\caption@setfont` `\caption@setfont{<command>}{<keyval-list of names>}`

Selecting a caption font means saving all the code snippets (in #1). Because we use `\setkeys` recursive here we need to put this into an extra group and collect all the code snippets in `\caption@tempa` first.

```

214 \newcommand*\caption@setfont[2]{%
215   \let\caption@tempa\@empty
216   \begingroup
217     \setkeys{caption@fnt}{#2}%
218   \endgroup
219   \expandafter\let\csname caption#1\endcsname\caption@tempa}

```

These are the pre-defined font code snippets.

```

220 \DeclareCaptionFont{default}{}

221 \DeclareCaptionFont{scriptsize}{\scriptsize}
222 \DeclareCaptionFont{footnotesize}{\footnotesize}
223 \DeclareCaptionFont{small}{\small}
224 \DeclareCaptionFont{normalsize}{\normalsize}
225 \DeclareCaptionFont{large}{\large}
226 \DeclareCaptionFont{Large}{\Large}

```

```

227 \DeclareCaptionFont{up}{\upshape}
228 \DeclareCaptionFont{it}{\itshape}
229 \DeclareCaptionFont{sl}{\slshape}
230 \DeclareCaptionFont{sc}{\scshape}
231 \DeclareCaptionFont{md}{\mdseries}
232 \DeclareCaptionFont{bf}{\bfseries}
233 \DeclareCaptionFont{rm}{\rmfamily}
234 \DeclareCaptionFont{sf}{\sffamily}
235 \DeclareCaptionFont{tt}{\ttfamily}

```

`\captionsize` The old versions 1.x of the caption package offered this command to setup the font size used for captions. We still do so old documents will work fine.

```
236 \providecommand\captionsize{}
```

```
237 \DeclareCaptionOption{size}{\caption@setfont{size}{#1}}% changed v3.0a
```

Vertical spaces before and after captions

`\abovecaptionskip` Usually these skips are defined within the document class, but some document classes
`\belowcaptionskip` don't do so.

```

238 \@ifundefined{abovecaptionskip}{%
239   \newlength\abovecaptionskip\setlength\abovecaptionskip{10\p@}}{}
240 \@ifundefined{belowcaptionskip}{%
241   \newlength\belowcaptionskip\setlength\belowcaptionskip{0\p@}}{}

```

```

242 \DeclareCaptionOption{aboveskip}{\setlength\abovecaptionskip{#1}}
243 \DeclareCaptionOption{belowskip}{\setlength\belowcaptionskip{#1}}
244 \DeclareCaptionOption{skip}{\setlength\abovecaptionskip{#1}}% new 3.0d

```

Positioning

These macros handle the right position of the caption. Note that the position is actually *not* controlled by the caption kernel options, but by the user (or the package) instead. The user can put the `\caption` command where ever he likes! So this stuff is only to give us a hint where to put the right skips, the user usually has to take care for himself that this hint actually matches the right position. The user can also try out the experimental setting `position=auto` which means that the caption package should try to guess the actual position of the caption for himself. (But in many cases, for example in longtables, this is doomed to fail, so it's not documented in the user part of the documentation.)

```
245 \DeclareCaptionOption{position}{\caption@setposition{#1}}
```

`\caption@setposition` Selecting the caption position means that we put `\caption@position` to the right value. Please do *not* use the internal macro `\caption@position` in your own package or document, but use the wrapper macro `\caption@iftop` instead.

```

246 \newcommand*\caption@setposition[1]{%
247   \caption@ifinlist{#1}{d,default}{%
248     \def\caption@position{\caption@defaultpos}%
249   }{\caption@ifinlist{#1}{t,top,above}{%
250     \let\caption@position\@firstoftwo
251   }{\caption@ifinlist{#1}{b,bottom,below}{%
252     \let\caption@position\@secondoftwo
253   }{\caption@ifinlist{#1}{a,auto}{%

```

```

254 \let\caption@position\@undefined
255 }{%
256 \PackageError{caption}{Undefined caption position `#1'}{\caption@eh}%
257 }}}}

```

`\caption@defaultpos` The default ‘position’ is ‘bottom’, this means that the (larger) skip will be typeset above the caption. This corresponds to the `\@makecaption` implementation in the standard L^AT_EX document classes.

```

258 %\caption@setdefaultpos{b}% default = bottom
259 \let\caption@defaultpos\@secondoftwo

```

```

\caption@iftop \caption@iftop{<true-code>}{<false-code>}
                (If \caption@position is set to ‘auto’ we assume a ‘bottom’ position.)
260 \newcommand\caption@iftop{% bugfixed v3.0a, improved v3.0d
261 \ifx\caption@position\@undefined
262 \expandafter\@secondoftwo
263 \else
264 \expandafter\caption@position
265 \fi}

```

`\caption@fixposition` This macro checks if the ‘position’ is set to ‘auto’. If yes, `\caption@autoposition` will be called to set `\caption@position` to a proper value we can actually use.

```

266 \newcommand\caption@fixposition{%
267 \ifx\caption@position\@undefined
268 \caption@autoposition
269 \fi}

```

`\caption@autoposition` We guess the actual position of the caption by checking `\prevdepth`.

```

270 \newcommand\caption@autoposition{% bugfixed v3.0a
271 \ifvmode
272 \ifodd\caption@debug\relax
273 \edef\caption@tempa{\the\prevdepth}%
274 \PackageInfo{caption}{\protect\prevdepth=\caption@tempa}%
275 \fi
276 %
277 % \caption@setposition{\ifdim\prevdepth>-\p@ b\else t\fi}%
278 \ifdim\prevdepth>-\p@
279 \let\caption@position\@secondoftwo
280 \else
281 \let\caption@position\@firstoftwo
282 \fi
283 \else
284 \ifodd\caption@debug\relax
285 \PackageInfo{caption}{no \protect\prevdepth}%
286 \fi
287 %
288 % \caption@setposition{b}%
289 \let\caption@position\@secondoftwo
290 \fi}

```

Hooks

```

\AtBeginCaption \AtBeginDocument {<code>}
\AtEndCaption \AtEndDocument {<code>}

```

These hooks can be used analogous to `\AtBeginDocument` and `\AtEndDocument`.

```
291 \newcommand\caption@beginhook{}
292 \newcommand\caption@endhook{}
293 \newcommand\AtBeginCaption{\l@addto@macro\caption@beginhook}
294 \newcommand\AtEndCaption{\l@addto@macro\caption@endhook}
```

Miscellaneous options

```
295 \DeclareCaptionOption{parskip}[5pt]{\AtBeginCaption{\setlength\parskip{#1}}}
296 \DeclareCaptionOption{listof}{\caption@setbool{lof}{#1}}% new v3.0b
297 \DeclareCaptionOption{singlelinecheck}[1]{\caption@setbool{slc}{#1}}
298 \DeclareCaptionOption{strut}{\caption@setbool{strut}{#1}}% new v3.0d
299 \DeclareCaptionOption{debug}{\def\caption@debug{#1}}
```

Initialization of parameters

```
300 \captionsetup{style=default,position=default,listof=1,strut=1,debug=0}

\ifcaption@star If the starred form of \caption is used, this will be set to true. (Note: This will be
replaced by \caption@iflabel in future versions of the caption package, so I can
use \caption@setbool so set this value.)
301 \newif\ifcaption@star
```

Typesetting the caption

```
\caption@make \caption@make{<float name>}{<ref. number>}{<text>}
302 \newcommand\caption@make[2]{%
303 \caption@@make{\caption@lfmt{#1}{#2}}}
```

```
\caption@@make \caption@@make{<caption label>}{<caption text>}
304 \newcommand\caption@@make[2]{%
305 % \beginngroup
306 \caption@beginhook
307 \caption@calcmargin

Special single-line treatment (Improvement v3.0d: moved to here)
```

```
308 \caption@ifslc{%
309 \ifx\caption@sls\@empty\else
310 \caption@startslc
311 \setbox\@tempboxa\hbox{\caption@@make{#1}{#2}}%
312 \ifdim\wd\@tempboxa >\captionwidth
313 \caption@endslc
314 \else
315 \caption@endslc
316 \caption@esetup\caption@sls
317 \caption@calcmargin
318 \fi
319 \fi}{}}%
```

Bugfix v3.0d: Use `\@tempdima` instead of `\captionmargin`, `\ifdim` added (04-10-26)

```
320 \@tempdima\captionmargin
321 \caption@ifh{\advance\@tempdima by \captionindent}%
```

```

322 \ifdim\@tempdima=\z@\else
323 \hskip\@tempdima
324 \fi

```

Bugfix v3.0d: Use \@tempdima instead of \captionwidth (04-10-26)

```

325 \@tempdima\captionwidth
326 \caption@ifh{\advance\@tempdima by -\captionindent}%
327 \caption@startbox\@tempdima

```

Bugfix v3.0b: \ifdim added (04-05-05)

Bugfix v3.0d: \leavevmode added (05/02/09)

Improvement v3.0d: \caption@ifh (05/02/09)

```

328 \caption@ifh{%
329 \ifdim\captionindent=\z@
330 \leavevmode
331 \else
332 \hskip-\captionindent
333 \fi}%

```

Bugfix v3.0d: \strut moved from here to \caption@@@make

```

334 \caption@@@make{#1}{#2}%
335 \caption@endbox

```

Bugfix v3.0d: This \hskip added

```

336 \ifdim\captionmargin=\z@\else
337 \hskip\captionmargin
338 \fi
339 \caption@endhook
340 % \endgroup
341 \global\caption@starfalse}

```

\caption@calcmargin Calculate \captionmargin & \captionwidth, so both contain valid values.

```

342 \newcommand\caption@calcmargin{%
343 \ifcaption@width
344 \captionmargin\hsize
345 \advance\captionmargin by -\captionwidth
346 \divide\captionmargin by 2
347 \else
348 \captionwidth\hsize
349 \advance\captionwidth by -2\captionmargin
350 \fi
351 %
352 \ifodd\caption@debug\relax
353 \PackageInfo{caption}{\protect\hsize=\the\hsize,
354 \protect\margin=\the\captionmargin,
355 \protect\width=\the\captionwidth}%
356 \fi}

```

\caption@startslc Re-define anything which would disturb the single line check

Bugfix v3.0b: re-definition of \label & \@footnotetext was missing here

Improvement v3.0b: re-define \stepcounter instead of \footnote (mark)

Improvement v3.0d: \let\caption@hj\relax added

```

357 \newcommand\caption@startslc{%

```

```

358 \begingroup
359 \let\label\@gobble\let\@footnotetext\@gobble
360 \def\stepcounter##1{\advance\c@##1\endcsname\@ne\relax}%
361 \let\caption@hj\relax}
362 \newcommand\caption@endslc{%
363 \endgroup}

\caption@startbox These macros start and end the box which surrounds the caption paragraph.
\caption@endbox 364 \newcommand*\caption@startbox[1]{\vbox\bgroup\hsize#1}%
365 %\newcommand*\caption@startbox[1]{\vbox\bgroup\setlength\hsize{#1}\@parboxrestore}%
366 \newcommand*\caption@endbox{\egroup}
367 %\newcommand*\caption@endbox{\@finalstrut\strutbox\@@par\egroup}

\caption@@@make \caption@@@make{<caption label>}{<caption text>}
This one finally typesets the caption paragraph, without margin and indentation.
368 \newcommand\caption@@@make[2]{%
Empty text? Then use no caption label separator.
369 \caption@ifempty{#2}{% changed v3.0e
370 \let\caption@lsep\relax
371 \let\caption@ifstrut\@secondoftwo % added v3.0e
372 }%

Take care that \captionparindent and \captionhangindent will be used to
typeset the paragraph.
373 \def\caption@@par{%
374 \parindent\captionparindent\hangindent\captionhangindent}%
375 \@setpar{\@@par\caption@@par}\caption@@par

Finally the caption will be typeset.
376 \caption@hj\captionsize\captionfont

Bugfix v3.0e: Handling of \ifcaption@star changed
377 \caption@fmt{\ifcaption@star\else{\captionlabelfont#1}\fi}%
378 {\ifcaption@star\else{\captionlabelfont\caption@lsep}\fi}%
379 {\captiontextfont

Bugfix v3.0d: Use some kind of \@startstrut\strutbox instead of \strut (04-
12-16)
380 \caption@ifstrut{\vrule\@height\ht\strutbox\@width\z@}{}%

Bugfix v3.0b: \allowhyphens added (04-05-06)
381 \nobreak\hskip\z@skip
382 #2%

Bugfix v3.0d: \@finalstrut\strutbox added (05-01-23)
383 % \caption@ifstrut{\vrule\@height\z@\@depth\dp\strutbox\@width\z@}{%
384 \caption@ifstrut{\@finalstrut\strutbox}{}%
385 \par}}

\caption@ifempty \caption@ifempty{<text>}{<if-clause>}
(new v3.0e, 05/05/05)
386 \newcommand\caption@ifempty[1]{%
387 \def\caption@tempa{#1}%
388 \def\caption@tempb{\ignorespaces}%

```

```

389 \ifx\caption@tempa\caption@tempb
390 \let\caption@tempa\@empty
391 \fi
392 \ifx\caption@tempa\@empty
393 \expandafter\@firstofone
394 \else
395 \expandafter\@gobble
396 \fi}

```

11.2 Main package

Identification

```

397 \NeedsTeXFormat{LaTeX2e}[1994/12/01]
398 \ProvidesPackage{caption}[2005/06/26 v3.0f Customising captions (AS)]

```

Loading the caption kernel

```

399 \RequirePackage{caption3}
400 \DeclareCaptionOption{type}{\def\@capttype{#1}}% new v3.0d

```

Float names

```
\caption@floatname \caption@floatname{<type>}
```

Usually all float names (which partly build the caption label) follow the same naming convention. But some packages (for example the float package) do not, so we use this wrapper macro which can be extended later on.

```
401 \newcommand*\caption@floatname[1]{\@nameuse{#1name}}
```

Support for figure and table

```

402 \DeclareCaptionOption*{figureposition}{\captionsetup[figure]{position=#1}}% new v
403 \DeclareCaptionOption*{tableposition}{\captionsetup[table]{position=#1}}% new v

```

Configuration files

```

404 \DeclareCaptionOption{config}[caption]{%
405 \InputIfFileExists{#1.cfg}{\typeout{*** Local configuration file
406 \caption#1.cfg used ***}}%
407 \PackageWarning{caption}{Configuration
408 file #1.cfg not found}}

```

Compatibility options (caption v1.x)

```

409 \DeclareCaptionOption*{normal}[]{\caption@setformat{normal}}
410 \DeclareCaptionOption*{isu}[]{\caption@setformat{hang}}
411 \DeclareCaptionOption*{hang}[]{\caption@setformat{hang}}
412 \DeclareCaptionOption*{center}[]{\caption@setjustification{centering}}
413 \DeclareCaptionOption*{anne}[]{\caption@setjustification{centerlast}}
414 \DeclareCaptionOption*{centerlast}[]{\caption@setjustification{centerlast}}

415 \DeclareCaptionOption*{scriptsize}[]{\def\captionfont{\scriptsize}}
416 \DeclareCaptionOption*{footnotesize}[]{\def\captionfont{\footnotesize}}
417 \DeclareCaptionOption*{small}[]{\def\captionfont{\small}}
418 \DeclareCaptionOption*{normalsize}[]{\def\captionfont{\normalsize}}
419 \DeclareCaptionOption*{large}[]{\def\captionfont{\large}}
420 \DeclareCaptionOption*{Large}[]{\def\captionfont{\Large}}

```

```

421 \DeclareCaptionOption*{up}[]{\l@addto@macro\captionlabelfont\upshape}
422 \DeclareCaptionOption*{it}[]{\l@addto@macro\captionlabelfont\itshape}
423 \DeclareCaptionOption*{sl}[]{\l@addto@macro\captionlabelfont\slshape}
424 \DeclareCaptionOption*{sc}[]{\l@addto@macro\captionlabelfont\scshape}
425 \DeclareCaptionOption*{md}[]{\l@addto@macro\captionlabelfont\mdseries}
426 \DeclareCaptionOption*{bf}[]{\l@addto@macro\captionlabelfont\bfseries}
427 \DeclareCaptionOption*{rm}[]{\l@addto@macro\captionlabelfont\rmfamily}
428 \DeclareCaptionOption*{sf}[]{\l@addto@macro\captionlabelfont\sffamily}
429 \DeclareCaptionOption*{tt}[]{\l@addto@macro\captionlabelfont\ttfamily}

430 \DeclareCaptionOption*{nooneline}[]{\caption@setbool{slc}{0}}

431 \caption@setbool{ruled}{0}
432 \DeclareCaptionOption*{ruled}[]{\caption@setbool{ruled}{1}}

```

Generic package support

`\DeclareCaptionPackage` Each single package support can be switched on or off by using the appropriate option. By default all of them are enabled.

```

433 \newcommand*\DeclareCaptionPackage[1]{%
434   \caption@setbool{pkt@#1}{1}%
435   \DeclareCaptionOption*{#1}{\caption@setbool{pkt@#1}{##1}}
436 \AtEndOfPackage{\let\DeclareCaptionPackage\@undefined}

```

```

\caption@ifpackage \caption@ifpackage{<package name>}{<package macro>}{<package code>}

437 \newcommand\caption@ifpackage[3]{%
438   \caption@ifbool{pkt@#1}{%
439     \@ifundefined{#2}%
440     {\let\next\AtBeginDocument}%
441     {\let\next\@firstofone}%
442   }{%
443     \let\next\@gobble
444   }%
445 %
446   \ifodd\caption@debug\relax
447   \edef\caption@tempa{%
448     \caption@ifbool{pkt@#1}{%
449       \@ifundefined{#2}{AtBeginDocument}{firstofone}%
450     }{gobble}}%
451   \PackageInfo{caption}{#1 = \caption@ifbool{pkt@#1}{1}{0} %
452     (\@ifundefined{#2}{not }{loaded -> \caption@tempa})%
453   \fi
454 %
455   \@nameundef{caption@ifpkt@#1}%   bugfixed v3.0a
456 %
457   \next{%
458     \expandafter\ifx\csname #2\endcsname\relax
459     \else
460       #3
461     \fi}
462 \AtEndOfPackage{\let\caption@ifpackage\@undefined}

```

These are the packages we support:

```

463 \DeclareCaptionPackage{caption}

```

```

464 \DeclareCaptionPackage{float}
465 \DeclareCaptionPackage{floatrow}
466 \DeclareCaptionPackage{hyperref}
467 \DeclareCaptionPackage{hypcap}
468 \DeclareCaptionPackage{listings}
469 \DeclareCaptionPackage{longtable}
470 \DeclareCaptionPackage{rotating}
471 \DeclareCaptionPackage{sidecap}
472 \DeclareCaptionPackage{supertabular}

```

`\ProcessOptionsWithKV` We process our options using the keyval package.

```

473 \def\ProcessOptionsWithKV#1{% bugfixed v3.0a
474   \let\@tempc\relax
475   \let\caption@tempa\@empty
476   \@for\CurrentOption:=\@classoptionslist\do{%
477     \ifundefined{KV@#1@\CurrentOption}%
478       {}%
479     {%
480       \edef\caption@tempa{\caption@tempa,\CurrentOption,}%
481       \@expandtwoargs\@removeelement\CurrentOption
482       \@unusedoptionlist\@unusedoptionlist
483     }%
484   }%
485   \edef\caption@tempa{%
486     \noexpand\setkeys{#1}{%
487       \caption@tempa\@optionlist{\@currname.\@currentx}%
488     }%
489   }%
490   \caption@tempa

```

Bugfix, see <400D360C.9678329F@gmx.net> for details

```

491   \let\CurrentOption\@empty
492   \@AtEndOfPackage{\let\@unprocessedoptions\relax}}

493 \ProcessOptionsWithKV{caption}
494 \let\ProcessOptionsWithKV\@undefined

495 \caption@ifbool{pkt@caption}{}{\endinput}
496 \@nameundef{caption@ifpkt@caption}

```

Usefull stuff

`\captionof` `\captionof(*){<type>}[<lst.entry>]{<heading>}`

```

497 \def\captionof{\@ifstar{\caption@of{\caption*}}{\caption@of\caption}}
498 \newcommand*\caption@of[2]{\def\@captive{#2}#1}

```

`\ContinuedFloat` `\ContinuedFloat`

```

499 \providecommand\ContinuedFloat{%
500   \ifx\@captive\@undefined
501     \@latex@error{\noexpand\ContinuedFloat outside float}\@ehd
502   \else
503     \addtocounter\@captive\m@ne
504     \caption@ContinuedFloat\@captive
505   \fi}%

```

```

\caption@ContinuedFloat
caption@resetContinuedFloat 506 \let\caption@ContinuedFloat\@gobble
507 \let\caption@resetContinuedFloat\@gobble

```

Internal helpers

```

\caption@begin \caption@begin{<type>} (changed in v3.0b+v3.0e)
508 \newcommand*\caption@begin[1]{%
509 \caption@resetContinuedFloat{#1}%
510 \begingroup
511 \caption@setfloattype{#1}%
512 % \caption@setfnum{#1}%
513 \ifx\caption@lfmt\caption@lfmt@default\else
514 \@namedef{fnum{#1}}{%
515 \caption@lfmt{\caption@floatname{#1}}{\@nameuse{the#1}}}%
516 \fi
517 \caption@fixposition
518 \global\let\caption@fixedposition\caption@position
519 \caption@@begin{#1}}

\caption@beginex \caption@beginex{<type>}{<list entry>}
520 \newcommand*\caption@beginex[1]{%
521 \caption@begin{#1}%
522 \caption@preparelof}

\caption@end \caption@end
523 \newcommand*\caption@end{%
524 \caption@@end
525 \endgroup
526 \let\caption@position\caption@fixedposition}

\caption@setfloattype A macro for setting up the right float type within \@caption, \LT@makecaption
etc. Usually this is equivalent to \caption@settype but I made it an own macro so I
can extend it later on, for example if the float package is loaded.
527 \let\caption@setfloattype\caption@settype% new v3.0a

\caption@letfloattype \caption@letfloattype{<type>}{<extra code>}
(new in v3.0b, additional argument in v3.0e)
528 \newcommand*\caption@letfloattype[2]{%
529 \def\caption@setfloattype##1{%
530 \caption@settype{##1}#2\caption@settype{#1}}}%

\caption@preparelof \caption@preparelof{<list entry>}
531 \newcommand*\caption@preparelof[1]{% changed v3.0b
532 \caption@iflof%
533 {\def\caption@tempa{#1}}%
534 {\let\caption@tempa\@empty}%
535 \ifx\caption@tempa\@empty
536 \def\addcontentsline##1##2##3{%
537 \fi}

```

```

\caption@@begin \caption@@begin{<type>}
\caption@@end \caption@@end
538 \let\caption@@begin\@gobble % new v3.0a
539 \let\caption@@end\@empty % new v3.0a

```

Caption support

Some packages (like the `hyperref` package for example) redefines `\caption` and `\@caption`, too, but without chaining to the previous definition. So we have to use `\AtBeginDocument` here, so we can make sure our definition don't get lost.

```

540 \AtBeginDocument{%
541   \let\caption@old\caption
542   \let\caption@@old\@caption
543   \@ifundefined{cc@caption}{%

\caption Define \caption* ...
(07/18/03: \global added, so this works with sidecap) (05/22/05: \ContinuedFloat
added)
544   \def\caption{\caption@caption\caption@old}%
545   \def\caption@caption#1{%
546     \@ifstar{\ContinuedFloat\global\caption@startrue #1[]}{#1}}%

\@caption Define \caption[] {...} ...
547   \long\def\@caption#1[#2]#3{%
548     \caption@beginex{#1}{#2}%
549     \caption@@old{#1}[{#2}]{#3}%
550     \caption@end}%

551   }{%

Minimum captcont package support (bugfixed v3.0c, 04-07-15)
552   \PackageInfo{caption}{captcont package v2.0 detected}%
553   \def\caption@caption#1{#1}% added v3.0c
554   }%
555 }

\@makecaption \@makecaption{<label>}{<text>}

```

Original code (from latex/base/classes.dtx):

```

\long\def\@makecaption#1#2{%
  \vskip\abovecaptionskip
  \sbox\@tempboxa{#1: #2}%
  \ifdim \wd\@tempboxa >\hsize
    #1: #2\par
  \else
    \global \@minipagefalse
    \hb@xt@\hsize{\hfil\box\@tempboxa\hfil}%
  \fi
  \vskip\belowcaptionskip}

```

```

556 \renewcommand\@makecaption[2]{%
557   \caption@iftop{\vskip\belowcaptionskip}{\vskip\abovecaptionskip}%
558   \ifnum\caption@debug>1 %
559     \llap{\$ \caption@iftop\downarrow\uparrow$ }%
560   \fi
561   \caption@@make{#1}{#2}%
562   \caption@iftop{\vskip\abovecaptionskip}{\vskip\belowcaptionskip}}

```

KOMA-script classes support

(new in 3.0a)

```

563 \AtBeginDocument{\let\scr@caption\caption}

```

float package support

The float package usually do not use the LaTeX kernel command `\caption` to typeset the caption but `\float@caption` instead. (`\caption` will only be used if the float is re-styled with `\restylefloat*`.)

The main two things `\float@caption` is doing different are:

- The caption will be typeset inside a savebox called `\@floatcapt` so it can be placed above or below the float contents afterwards.
- `\@makecaption` will not be used to finally typeset the caption. Instead `\@fs@capt` will be used which definition is part of the float style. (Note that `\@fs@capt` will not typeset any vertical space above or below the caption; instead this space will be typeset by the float style code itself.)

So our main goal is to re-define `\float@caption` so our macro `\caption@@make` will be used instead of `\@fs@capt`.

To allow different caption styles for different float styles we will also determine the current float style (e.g. ‘ruled’) at run time and setup a caption style (and additional settings) with the same name, if defined.

`\caption@setfloatposition` First of all we provide a macro which converts `\@fs@iftopcapt` (which is part of a float style and controls where the caption will be typeset, above or below the float contents) to our `position=` setting. Since the spacing above and below the caption will be done by the float style and *not* by us this sounds quite useless. But in fact it isn’t, since some packages based on the caption package (like the subfig package) could have an interest for this information and therefore use the `\caption@iftop` macro we provide in our kernel. Furthermore we need this information for ourself in `\captionof` which uses `\@makecaption` to finally typeset the caption.

```

564 \def\caption@setfloatposition{%
565   \caption@setposition{\@fs@iftopcapt t\else b\fi}}

566 \caption@ifpackage{float}{@float@setevery}{%
567   \PackageInfo{caption}{float package v1.3 (or newer) detected}%

```

Since `\float@caption` puts the float contents into a savebox we need a special version of `\captionof` which ‘unfolds’ this box afterwards, so the caption actually gets typeset. Furthermore we have to typeset the spacing above and below the caption for ourself, since this space is not part of the box.

Please note that this version of `\captionof` only works *outside* floating environments defined with the float package, so for example a `\captionof{Program}` used within a ‘standard’ figure or a minipage will work fine, but not within a re-styled figure or an Example environment defined with `\newfloat`. (We don’t check for this so you’ll get wired errors if you try to do so!)

`\caption@of@float` Usually no special action is necessary, so we define `\caption@of@float` to `\@gobble`. We will redefine it later on to `\@firstofone` to activate the code which ‘unfolds’ the savebox.

```
568 \let\caption@of@float\@gobble
```

`\caption@of` If the float is defined by the float package (which means `\fst@<type>` is defined) we activate the special treatment for such captions typeset with `\captionof`. Furthermore we ‘execute’ this float style, so `\@fs@iftopcapt` is set to its proper value.

```
569 \renewcommand*\caption@of[2]{%
570   \ifundefined{fst@#2}{}{%
571     \let\caption@of@float\@firstofone
572     \@nameuse{fst@#2}\@float@setevery{#2}}%
573   \def\@capttype{#2}#1}%
```

`\float@caption` Our version of `\float@caption` nearly looks like our version of `\@caption`. The main differences are that `\@fs@capt` will be replaced by our `\caption@@make` and that the savebox called `\@floatcapt` will be unfolded if required. (See above)

```
574 \let\caption@@float\float@caption
575 \long\def\float@caption#1[#2]#3{%
576   \caption@beginex{#1}{#2}%
577   \let\@fs@capt\caption@@make
578   \caption@@float{#1}[[#2]][#3]%
579   \caption@of@float{%
580     \def\caption@@make##1##2{\unvbox\@floatcapt}%
581     \@makecaption{}{}}%
582   \caption@end}%
```

`\@float@setevery` `\@float@setevery{<float type>}` is provided by the float package; it’s called every time a floating environment defined with `\newfloat` or `\restylefloat` begins. We use this hook to do some adaptations and to setup the proper caption style (if defined) and additional settings declared with `\captionsetup[<float style>]`.

```
583 \let\caption@float@setevery\@float@setevery
584 \def\@float@setevery#1{%
585   \caption@float@setevery{#1}%
```

L^AT_EX and most packages use `\<type>`name to provide a macro for the float name – for example the command `\figurename` will usually contain the name of the floating environment figure:

```
\newcommand\figurename{Figure}
```

But the float package don’t follow this naming convention, it uses `\fname@<type>` instead. So we have to adapt `\caption@floatname` here, so our captions will be still ok.

```
586 \def\caption@floatname##1{\@nameuse{fname@#1}}%
```

`\newfloat` and `\restylefloat` saves the *actual* definition of `\@caption` or `\float@caption` in `\@float@c@⟨captype⟩` with `\let` (instead of using `\def`), so redefinitions of `\@caption` (and of course our redefinition of `\float@caption`) will never be used if the `\newfloat` or `\restylefloat` command takes place in front of the redefinitions provided by the `caption` or other packages like the `hyperref` package.

So here we determine if the user has used `\restylefloat` or `\restylefloat*` and bring `\@float@c@⟨captype⟩` up-to-date. This is quite easy: If `\@float@c@⟨captype⟩` is the same as the original or our own definition of `\float@caption`, the user has used `\restylefloat` (and `\float@caption` should be used), otherwise we assume he has used `\restylefloat*` (and `\@caption` should be used). (This test will fail if some other package re-defines `\float@caption`, too, so we have to assume that we are the only one.)

```

587 \expandafter\let\expandafter\caption@tempa\csname @float@c@#1\endcsname
588 \ifx\caption@tempa\float@caption
589 \else\ifx\caption@tempa\@caption
590 \else\ifx\caption@tempa\caption@@float
591 % \ifodd\caption@debug\relax
592 % \PackageInfo{caption}{\protect\@float@c@#1\space := \protect\float@caption}%
593 % \fi
594 \expandafter\let\csname @float@c@#1\endcsname\float@caption
595 \else
596 % \ifodd\caption@debug\relax
597 % \PackageInfo{caption}{\protect\@float@c@#1\space := \protect\@caption}%
598 % \fi
599 \expandafter\let\csname @float@c@#1\endcsname\@caption
600 \fi\fi\fi

```

If the floating environment is defined with `\newfloat` or `\restylefloat` (and *not* with `\restylefloat*`), `\@float@c@⟨type⟩` will now be identical to `\float@caption`.

```

601 \expandafter\ifx\csname @float@c@#1\endcsname\float@caption

```

First of all we set the caption position to its proper value. (See above definition of `\caption@setfloatposition`)

```

602 \caption@setfloatposition% changed v3.0b+f

```

Now we'll have to determine the current float style. This is not so easy because the only hint provided by the `float` package is the macro `\fst@⟨float type⟩` which points to the macro which represents the float style. So for example after

```

\floatstyle{ruled}
\newfloat{Program}{tbp}{lop}

```

`\fst@Program` will be defined as

```

\def\fst@Program{\fs@ruled} .

```

So here is what we do: We copy `\fst@⟨float type⟩` to `\caption@fst` and make it a string so we can gobble the first four tokens (= `\fs@`), so only the the name of the float style is left.

```

603 \expandafter\let\expandafter\caption@fst\csname fst@#1\endcsname
604 \edef\caption@fst{\noexpand\string\expandafter\noexpand\caption@fst}%
605 \edef\caption@fst{\noexpand\@gobblefour\caption@fst}%
606 % \edef\caption@fst{\caption@fst}%

```

`\caption@fst` now contains the float style (e.g. ‘ruled’) so we can use it to set the corresponding style (if defined) and additional options.

```
607      \caption@setstyle*\caption@fst
608      \caption@settype\caption@fst% new v3.0f
609      \fi}%
```

`\fs@plaintop` The float styles `plaintop` and `boxed` don’t use our `skip` which can be set with `skip=`
`\fs@boxed` : `plaintop` uses `\belowcaptionskip` instead of `\abovecaptionskip`, and
`boxed` uses a fixed space of 2pt. So we patch the according float style macros here.

```
610  \g@addto@macro\fs@plaintop{\def\@fs@mid{\vspace\abovecaptionskip\relax}}%
611  \g@addto@macro\fs@boxed{\def\@fs@mid{\kern\abovecaptionskip\relax}}%

612 }

613 \captionsetup[boxed]{skip=2pt}%
```

To emulate the ‘ruled’ definition of `\@fs@capt` we provide a caption style ‘ruled’ with appropriate options. But if the package option `ruled` was specified, we setup additional caption settings to emulate the behaviour of the caption package v1.x option `ruled` instead: The current caption settings will be used, but without margin and without ‘single-line-check’.

```
614 \caption@ifbool{ruled}{%
615   \captionsetup[ruled]{margin=0pt,singlelinecheck=0}% new v3.0f
616 }{% v3.0f: "strut=0" added
617   \DeclareCaptionStyle{ruled}{labelfont=bf,labelsep=space,strut=0}%
618 \let\caption@ifruled\@undefined
```

floatrow package support

The `floatrow` package is adapted for usage with the `caption` package. So the main work has already been done, there are only two little things we have to take care about.

```
619 \caption@ifpackage{floatrow}{\flrow@setlist}{%
620   \PackageInfo{caption}{floatrow package v0.1f (or newer) detected}%
```

`\caption@of` Captions typeset with `\captionof` should have the correct layout, so we have to ‘activate’ this layout here with `\flrow@setlist`.

(Please note that this version of `\captionof` has the same restrictions than the `\captionof` offered for floating environments defined with the `float` package, see above.)

```
621   \renewcommand*\caption@of[2]{%
622     \def\@captive{#2}\flrow@setlist{{#2}}#1}%
```

`\caption@floatname` The `floatrow` package follows the same naming convention as the `float` package; so we have to adapt `\caption@floatname` here, too.

```
623   \renewcommand*\caption@floatname[1]{%
624     \@nameuse{\@ifundefined{fname@#1}{#1name}{fname@#1}}}%

625 }
```

hyperref package support

When the hyperref package is used we have the problem that the usage of `\ContinuedFloat` will create duplicate hyperlinks – both `\@currentHlabel` and `\@currentHref` will be the same for the main float and the continued ones. So we have to make sure unique labels and references will be created each time. We do this by extending `\theHfigure` and `\theHtable`, so for continued floats the scheme

$$\langle type \rangle . \langle type \# \rangle . \langle continue \# \rangle$$

will be used instead of

$$\langle type \rangle . \langle type \# \rangle .$$

(This implementation follows an idea from Steven Douglas Cochran.)

Note: This does not help if `\Hy@naturalnamestrue` is set.

```
626 \caption@ifpackage{hyperref}{\theHfigure}{%
627   \PackageInfo{caption}{hyperref package v6.74m (or newer) detected}%
\caption@ContinuedFloat If \theH<type> is defined, we extend it with .<continue #>. Furthermore we set
\caption@resetContinuedFloat to \@gobble so the continuation counter will
not be reset to zero inside \caption.
628   \def\caption@ContinuedFloat#1{%
629     \@ifundefined{theH#1}{}{%
630       \ifundefined{c@CF@#1}{\newcounter{CF@#1}}{%
631         \stepcounter{CF@#1}%
632         \expandafter\l@addto@macro\csname theH#1\endcsname{.\arabic{CF@#1}}%
633         \let\caption@resetContinuedFloat\@gobble
634       }%
\caption@resetContinuedFloat If a continuation counter is defined, we reset it.
635   \def\caption@resetContinuedFloat#1{%
636     \@ifundefined{c@CF@#1}{}{\@stpel{CF@#1}}%
637 }
```

hypcap package support

When the hypcap package is used the following problems occur:

1. The hypcap package uses `\capstart`, `\hc@caption`, and `\hc@@caption` instead of `\caption` and `\@caption`. So we have to patch these macros, too.
2. `\caption` will be saved to `\hc@org@caption` when the hypcap package is loaded. We have to change this so our definition of `\caption` will always be used.
3. Both, `\capstart` and `\hc@@caption`, call `\hyper@makecurrent`. But since we offer `\ContinuedFloat` the float counters could have changed between these both calls! So we fix this by saving the hyperref reference (`= \@currentHref`) in `\capstart` and restoring it later on in `\hc@@caption`.

(This also fixes the problem that hypcap does not work if `\Hy@hypertextnamesfalse` is set. This come in handy; we set it locally to avoid duplicated hyperref labels which could occur if `\ContinuedFloat` will be used.)

4. `\capstart` will call `\H@refstepcounter` to increase the float number. This collides with a following `\ContinuedFloat`, too, so we have to move this call from here to `\caption`. (Since we set `\Hy@hypertextnamesfalse` we can do this without problems.)

```
638 \caption@ifpackage{hypcap}{hc@caption}{%
639   \PackageInfo{caption}{hypcap package v1.0 (or newer) detected}%
```

`\capstart` Here comes our version of `\capstart`:

```
640 \let\caption@capstart\capstart
641 \def\capstart{%
```

First of all we update `\hc@org@caption` to correct the problem that the `hypcap` package has saved an older definition of `\caption`.

```
642 \let\hc@org@caption\caption
```

Since we don't know the float counter yet (it could be changed with `\ContinuedFloat` afterwards!) we make sure `\H@refstepcounter` will not be used and `\Hy@hypertextnamesfalse` is set, so unique `hyperref` labels will be generated by the original definition of `\capstart`. Afterwards we save the reference which was generated by `\hyper@makecurrent`.

```
643 \begingroup
644 \let\H@refstepcounter\@gobble
645 \Hy@hypertextnamesfalse
646 \caption@capstart
647 \global\let\caption@currentHref\@currentHref
648 \endgroup
```

The `hypcap` package restores the previous definition of `\caption` inside `\hc@@caption`. But since we will call this inside a group later on (making this restauration non-working), we have to make this for ourself inside `\caption`. (This would not be necessary if `hypcap` would do this inside `\hc@caption` instead of `\hc@@caption`.)

Additionally we increase the float counter here (since we have suppressed this in `\capstart`) and use `\caption@caption` here, so `\caption*` will work as expected.

```
649 \def\caption{%
650   \let\caption\hc@org@caption
651   \H@refstepcounter\@capytype
652   \caption@caption\hc@caption}}%
```

`\hc@@caption` Here comes our version of `\hc@@caption`:

```
653 \let\caption@hc@@caption\hc@@caption
654 \long\def\hc@@caption#1[#2]#3{%
655   \caption@beginex{#1}{#2}%
```

Beside the usual `\caption@begin` and `\caption@end` stuff (to support local options etc.) we make sure our saved `hyperref` reference will be used.

```
656 \let\caption@hyper@makecurrent\hyper@makecurrent
657 \def\hyper@makecurrent\@capytype{%
658   \let\hyper@makecurrent\caption@hyper@makecurrent
659   \global\let\@currentHref\caption@currentHref}%
660 \caption@hc@@caption{#1}[{#2}]{#3}%
661 \caption@end}%
662 }
```

listings package support

```
663 \caption@ifpackage{listings}{\lst@MakeCaption}{%
664   \PackageInfo{caption}{listings package v1.2 (or newer) detected}%
665 %
666   \let\caption@lst@MakeCaption\lst@MakeCaption
667   \def\lst@MakeCaption#1{%
668     \let\caption@setfloattype\caption@settype
669     \def\caption@autoposition{\caption@setposition{#1}}%
670     \caption@begin{lstlisting}%
671       \caption@lst@MakeCaption{#1}%
672     \caption@end}%
673 %
674 }
```

longtable package support

```
675 \caption@ifpackage{longtable}{LT@makecaption}{%
676   \PackageInfo{caption}{longtable package v3.15 (or newer) detected}%
677 %
678 % Original code:
679 % \def\LT@makecaption#1#2#3{%
680 %   \LT@mcol\LT@cols c{\hbox to\z@{\hss\parbox[t]{LTcapwidth{%
681 %     % Based on article class "\@makecaption", "#1" is "\@gobble" in star
682 %     % form, and "\@firstofone" otherwise.
683 %     \sbox\@tempboxa{#1{#2: }#3}%
684 %     \ifdim\wd\@tempboxa>\hsize
685 %       #1{#2: }#3%
686 %     \else
687 %       \hbox to\hsize{\hfil\box\@tempboxa\hfil}%
688 %     \fi
689 %     \endgraf\vskip\baselineskip}%
690 %   \hss}}}%
691 %
692 % \def\LT@makecaption#1#2#3{%
693 %   \noalign{\vskip...}%
694 %
695 %   \LT@mcol\LT@cols c{\hbox to\z@{\hss\parbox[t]{\hsize{%
696 %
697     \caption@letfloattype{longtable}{% bugfixed v3.0e
698       \ifdim\LTcapwidth=4in \else
699         \caption@setwidth\LTcapwidth
700       \fi}%
701 %   \caption@setdefaultpos{t}% default = top
702 %   \let\caption@defaultpos\@firstoftwo% default = top
703 %   \def\caption@autoposition{% does not work within \end{last}foot!
704 %     \caption@setposition{\ifcase\LT@rows t\else b\fi}}%
705 %
706 %   \caption@begin{table}%
707 %
708 %   This skip has 2 purposes:
709 %   1. Correct the heighth of the \parbox[t]. Usual it's the height of
710 %     the very first line, but because of our extra skip it's always 0pt.
711 %   2. Correct \arraystretch, which usually also affect the longtable
712 %     caption. (If this is not requested, take \strutbox instead.)
```

```

713 %      NOTE: This is only a quick workaround, it has to be revised later on.
714 %
715 %      \vskip-\ht\@arstrutbox
716 %
717 %      \caption@iftop{\vskip\belowcaptionskip}{\vskip\abovecaptionskip}%
718 %      \let\caption@beginbox\caption@beginLTbox
719 %      \caption@startrue#1\caption@starfalse
720 %      \caption@@make{#2}{#3}\endgraf
721 %      \caption@iftop{\vskip\abovecaptionskip}{\vskip\belowcaptionskip}%
722 %      \caption@end}%
723 %
724 %      \hss}}}%
725 %
726 }

```

rotating package support

```

727 \caption@ifpackage{rotating}{@rotcaption}{%
728   \PackageInfo{caption}{rotating package v2.0 (or newer) detected}%
729 %
730   \let\caption@rot\rotcaption
731   \def\rotcaption{\caption@caption\caption@rot}%
732 %
733   \let\caption@@rot\@rotcaption
734   \long\def\@rotcaption#1[#2]#3{%
735     \caption@beginex{#1}{#2}%
736     \caption@@rot{#1}[[#2]]{#3}%
737     \caption@end}%
738 %
739 % Original code:
740 % \long\def\@makerotcaption#1#2{%
741 %   \setbox\@tempboxa\hbox{#1: #2}%
742 %   \ifdim \wd\@tempboxa > .8\vsiz
743 %     \rotatebox{90}{%
744 %       \begin{minipage}{.8\textheight}#1: #2\end{minipage}%
745 %     }\par
746 %   \else%
747 %     \rotatebox{90}{\box\@tempboxa}%
748 %   \fi
749 %   \hspace{12pt}%
750 % }
751 %
752 % \long\def\@makerotcaption#1#2{%
753 %   \rotatebox{90}{%
754 %     \begin{minipage}{.8\textheight}%
755 %       \caption@@make{#1}{#2}%
756 %     \end{minipage}%
757 %   }\par
758 %   \hspace{12pt}}%
759 %
760 }

```

sidecap package support

```

761 \caption@ifpackage{sidecap}{endSC@FLOAT}{%
762   \PackageInfo{caption}{sidecap package v1.4d (or newer) detected}%

```

```

763 %
764 % First of all, we let sidecap use an actual definition of \caption:
765 % (This is only required for version 1.5d of the sidecap package.)
766 %
767 \let\SC@caption=\caption
768 %
769 % Make \caption* and local settings (\captionsetup) work
770 %
771 \let\caption@SC@zfloat\SC@zfloat
772 \def\SC@zfloat#1#2#3[#4]{%
773 % #2 = 'figure' or 'table' => \SC@captype
774   \caption@SC@zfloat{#1}{#2}{#3}[#4]%
775 %
776   \global\let\SC@CAPsetup\@empty
777   \def\captionsetup##1{\g@addto@macro\SC@CAPsetup{,##1}}%
778 %
779   \let\caption@old\caption
780 %   \def\caption{\renewcommand\captionsetup[1]{}\caption@caption\caption@old}%
781   \def\caption{\caption@caption\caption@old}%
782 }%
783 %
784 % Before typesetting the caption, we set the captionmargin to zero
785 % because the extra margin is only disturbing here.
786 % (We don't need to take care about the caption position because
787 % the sidecap package set both \abovecaptionskip and \belowcaptionskip
788 % to a skip of zero anyway.)
789 % Furthermore \SC@justify will override the caption justification, if set.
790 %
791 % Very old version (1.4): \SC@justify is not defined
792 % Older versions (1.5): \SC@justify is \relax when not set
793 % Newer versions (1.6): \SC@justify is \@empty when not set
794 %
795 \let\caption@endSC@FLOAT\endSC@FLOAT
796 \def\endSC@FLOAT{%
797 % (Note that \@captype isn't defined so far, this will be done inside
798 % the original definition of \endSC@FLOAT.)
799 % We set \@captype already here, so \captionsetup will
800 % work with \@captype-based options, too. (new v3.0d)
801   \let\@captype\SC@captype
802   \caption@esetup\SC@CAPsetup
803 %
804   \caption@letfloattype{SC\@captype}{% bugfixed v3.0e
805     \caption@setmargin\z@
806     \@ifundefined{SC@justify}{}{%
807       \ifx\SC@justify\@empty\else
808         \let\caption@hj\SC@justify
809         \let\SC@justify\@empty
810       \fi}}%
811 %
812   \long\def\caption@ifempty##1{% bugfix v3.0e
813     \ifx\SC@CAPtext\@empty
814       \expandafter\@firstofone
815     \else
816       \expandafter\@gobble

```

```

817         \fi}%
818 %
819         \caption@endSC@FLOAT}%
820 %
821 }

```

supertabular package support

```

822 \def\caption@setSTposition{%
823   \caption@setposition{\if@topcaption t\else b\fi}}
824 %
825 \caption@ifpackage{supertabular}{ST@caption}{%
826   \PackageInfo{caption}{supertabular package detected}%
827 %
828 % Improvement v3.0e: \topcaption* and \bottomcaption*
829   \let\caption@tablecaption\tablecaption
830   \def\tablecaption{\caption@caption\caption@tablecaption}%
831 %
832 % Original code:
833 % \long\def\ST@caption#1[#2]#3{\par%
834 %   \addcontentsline{\csname ext@#1\endcsname}{#1}%
835 %     {\protect\numberline{%
836 %       \csname the#1\endcsname}{\ignorespaces #2}}}
837 %   \begingroup
838 %     \@parboxrestore
839 %     \normalsize
840 %     \if@topcaption \vskip -10\p@ \fi
841 %     \@makecaption{\csname fnum@#1\endcsname}{\ignorespaces #3}\par
842 %     \if@topcaption \vskip 10\p@ \fi
843 %   \endgroup}
844 %
845   \let\caption@ST\ST@caption
846   \long\def\ST@caption#1[#2]#3{\par%   bugfixed v3.0a
847     \caption@letfloattype{supertabular}{}}%
848   \let\caption@fixposition\caption@setSTposition
849   \caption@beginex{#1}{#2}%
850     \addcontentsline{\csname ext@#1\endcsname}{#1}%
851       {\protect\numberline{%
852         \csname the#1\endcsname}{\ignorespaces #2}}}%
853     \@parboxrestore
854     \normalsize
855     \@makecaption{\csname fnum@#1\endcsname}{\ignorespaces #3}\par
856   \caption@end}%
857 %
858 }

```

References

- [1] Frank Mittelbach and Michel Goossens: *The L^AT_EX Companion (2nd. Ed.)*, Addison-Wesley, 2004.
- [2] Anselm Lingnau: *An Improved Environment for Floats*, 2001/11/08
- [3] Olga Lapko: *The floatrow package documentation*, 2005/05/22
- [4] Sebastian Rahtz: *Hypertext marks in L^AT_EX*, 2003/11/30
- [5] Heiko Oberdiek: *The hypcap package – Adjusting anchors of captions* 2001/08/27
- [6] Carsten Heinz: *The Listings Package*, 2004/02/13
- [7] David Carlisle: *The longtable package*, 2000/10/22
- [8] Sebastian Rahtz and Leonor Barroca: *A style option for rotated objects in L^AT_EX*, 1997/09/26
- [9] Rolf Niepraschk und Hubert Gäßlein: *The sidecap package*, 2003/06/06
- [10] Steven D. Cochran: *The subfig package*, 2004/01/16
- [11] Johannes Braams und Theo Jurriens: *The supertabular environment*, 2002/07/19